KOÇ UNIVERSITY

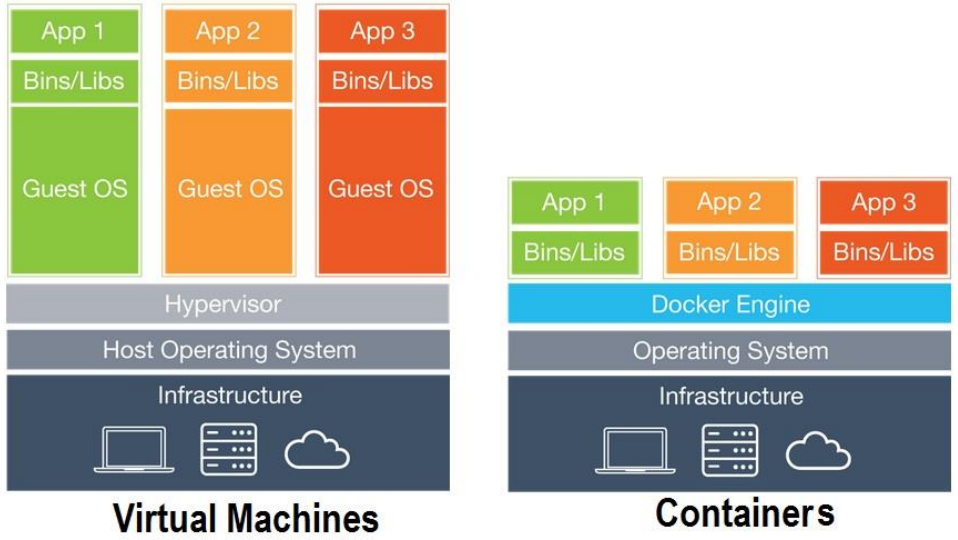
COMP 350
Introduction to DevOps

Lecture 7
Containers and Docker

Hakan Ayral

What is Docker?

- Docker is a set of platform as a service (PaaS) products that use OS-level virtualization to deliver software in packages called **containers**.
- Containers are isolated from one another and bundle their own software, libraries and configuration files; they can communicate with each other through well-defined channels.
- Because all of the containers share the services of a single operating system kernel, they use fewer resources than virtual machines.



What is Docker?

- Docker can package an application and its dependencies in a virtual container that can run on any OS/computer.
  - This enables the application to run in a variety of locations, such as on-premises, in a public cloud, and/or in a private cloud.
- When running on Linux, Docker uses the resource isolation features of the Linux kernel (*such as cgroups and kernel namespaces*) and a union-capable file system (*such as OverlayFS*) to allow containers to run within a single Linux instance, avoiding the overhead of starting and maintaining virtual machines.
- Because Docker containers are lightweight, a single server or virtual machine can run several containers simultaneously.
  - A 2018 analysis found that a typical Docker use case involves running eight containers per host, and that a quarter of analyzed organizations run 18 or more per host.

source: https://en.wikipedia.org/wiki/Docker_(software)

Components

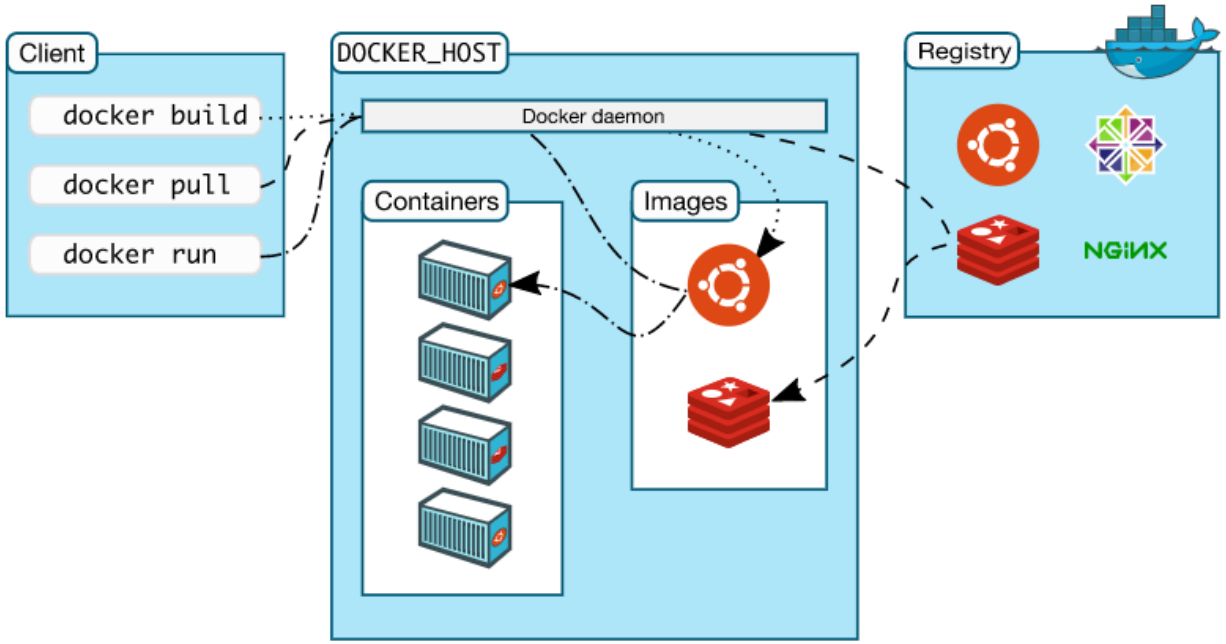
The Docker software as a service offering consists of three components:

- **Software**:
  - The **Docker daemon**, called **dockerd**, is a persistent process that manages Docker containers and handles container objects. The daemon listens for requests sent via the Docker Engine API.
  - The **Docker client** program, called **docker**, provides a command-line interface, CLI, that allows users to interact with Docker daemons.
- **Objects**: Docker objects are various entities used to assemble an application in Docker. The main classes of Docker objects are **images**, **containers**, and **services**.
  - A **Docker container** is a standardized, encapsulated environment that runs applications. A container is managed using the Docker API or CLI.
  - A **Docker image** is a read-only template used to build containers. Images are used to store and ship applications.
  - A **Docker service** allows containers to be scaled across multiple Docker daemons. The result is known as a swarm, a set of cooperating daemons that communicate through the Docker API.
- **Registries**: A Docker registry is a repository for Docker images. Docker clients connect to registries to download ("pull") images for use or upload ("push") images that they have built. Registries can be public or private. Docker Hub is the default registry where Docker looks for images.

source: https://en.wikipedia.org/wiki/Docker_(software)

Docker Client, Daemon and Registry

- Docker **Client** and Docker **Daemon** can be on the same physical computer or they can be on different computers and communicate through network.
- **Registry** is almost always a server on the internet or LAN.
- You can have multiple instances of the same **image** running as different **containers** (*e.g. having multiple MySql servers running*)



Why the different package names?

- Older versions of the Docker binary were called docker or docker-engine or docker-io
  - **docker-io** package is still the name used by Debian/Ubuntu for the docker release provided on their official repos.
  - **docker-ce** is a certified release provided directly by docker.com and can also be built from source.
- Main reason for using the name **docker-io** on Debian/Ubuntu platform was to avoid a name conflict with docker system-tray binary which used to have the package name "docker" (*it is now renamed as "wmdocker"*).
- Docker has an enterprise version (EE) and a free community Edition version(CE), the "-ce" on "**docker-ce**" comes from "community edition"

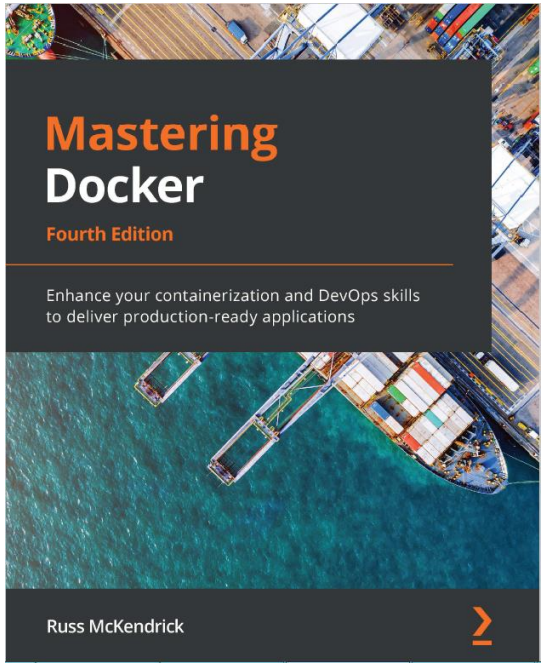
Why the different package names?

- docker-ce is provided by docker.com, docker.io is provided by Debian.
- This means you can install docker.io directly from your linux distro repository, while for docker-ce you have to first add the official repository from docker.com.
- More importantly, however, although both packages provide properly released versions of Docker, they have a very different internal structure:
  - **docker.io** does it the Debian (or Ubuntu) way: Each external dependency is a separate package that can and will be updated independently.
  - **docker-ce** does it the Golang way: All dependencies are pulled into the source tree before the build and the whole thing forms one single package afterwards. So you always update docker with all its dependencies at once.

source: https://stackoverflow.com/questions/45023363/what-is-docker-io-in-relation-to-docker-ce-and-docker-ee/45023650

Reference Material

- During the lectures on Docker the information on the slides will come from various sources but if you feel the need to use a reference material you can refer to Mastering Docker 4th ed. by Russ McKendrick.



Uninstalling previous versions

- If you are going to install Docker Community Edition (docker-ce from docker.com) you may need to remove older binaries especially if you have previously installed docker from your distro repo instead of docker.com repo.

```
$ sudo apt-get remove docker docker-engine docker.io containerd runc
```

  - In our case this is not necessary because we haven't installed any version of Docker to our VM yet.

- The docker-ce binaries will tend to be the latest versions, while the version provided by your distro repo may be one version behind.

Installing Docker from debian/ubuntu package repository

- You can install docker from your Linux distribution's repository with `apt-get install docker.io` if you are using a distribution which is based on Debian.
  - Note:
    - This download is over 300MB.
    - You might need to do `apt-get update` first depending on when you last updated your local metadata cache!

```
hayral@Computer1:~$ sudo apt-get install docker.io
[sudo] password for hayral:
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following additional packages will be installed:
  bridge-utils cgroupfs-mount containerd git git-man liberror-perl pigz runc
  ubuntu-fan
Suggested packages:
  aufs-tools debootstrap docker-doc rinse zfs-fuse | zfsutils git-daemon-run
  | git-daemon-sysvinit git-doc git-el git-email git-gui gitk gitweb git-cvs
  git-mediawiki git-svn
The following NEW packages will be installed:
  bridge-utils cgroupfs-mount containerd docker.io git git-man liberror-perl
  pigz runc ubuntu-fan
0 upgraded, 10 newly installed, 0 to remove and 268 not upgraded.
Need to get 74,8 MB of archives.
After this operation, 372 MB of additional disk space will be used.
Do you want to continue? [Y/n]
```

Installing Docker from Docker repository

- First, update your existing list of packages:

```
sudo apt update
```

- Next, install a few prerequisite packages which let apt use packages over HTTPS:

```
sudo apt install apt-transport-https ca-certificates curl software-properties-common
```

- Then add the GPG key for the official Docker repository to your system:

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo apt-key add –
```

- Add the Docker repository to APT sources:

```
sudo add-apt-repository "deb [arch=amd64] https://download.docker.com/linux/ubuntu focal stable"
```

  - This will also update our package database with the Docker packages from the newly added repo.

Installing Docker from Docker repository

- Finally, install Docker:

```
sudo apt install docker-ce
```

- Docker should now be installed, the daemon started, and the process enabled to start on boot. Check that it's running:

```
sudo systemctl status docker
```

- The output should be similar to the following, showing that the service is active and running:

```
Output
● docker.service - Docker Application Container Engine
     Loaded: loaded (/lib/systemd/system/docker.service; enabled; vendor preset: enabled)
     Active: active (running) since Tue 2020-05-19 17:00:41 UTC; 17s ago
TriggeredBy: ● docker.socket
       Docs: https://docs.docker.com
   Main PID: 24321 (dockerd)
      Tasks: 8
     Memory: 46.4M
     CGroup: /system.slice/docker.service
             └─24321 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.soc
```

Starting the Service

- If you installed Docker form debian/ubuntu package repository, after installing docker you should start the docker service with "**`systemctl start`**" and set it to auto-start at each boot by "**`systemctl enable`**".

- or you can start the docker service manually with "**`systemctl start`**" everytime you are going to use it

```
hayral@Computer1:~$ sudo systemctl start docker
hayral@Computer1:~$ sudo systemctl enable docker
hayral@Computer1:~$
```

- Let's see if Docker has installed correctly by checking it's version:

```
hayral@Computer1:~$ docker --version
Docker version 20.10.2, build 20.10.2-0ubuntu1~20.04.2
hayral@Computer1:~$
```

Interlude
systemd, systemctl, and services

- Used to introspect and control the state of the "systemd" system and service manager.

most common **init** implementation

systemd

Article Talk

From Wikipedia, the free encyclopedia

systemd is a software suite for system and service management on Linux[7] built to unify service configuration and behavior across Linux distributions.[8] Its main component is an init system used to bootstrap user space and manage user processes. It also provides replacements for various daemons and utilities, including device management, login management, network connection management, and event logging. The name *systemd* adheres to the Unix convention of naming daemons by appending the letter *d*,[9] and also plays on the French phrase *Système D* (a person's ability to quickly adapt and improvise in the face of problems).[10]

Since 2015, nearly all Linux distributions have adopted systemd. It has been praised by developers and users of distributions that adopted it for providing a stable, fast out-of-the-box solution for issues that had existed in the Linux space for years.[11][12][13] At the time of its adoption, it was the only parallel boot and init system offering centralized management of processes, daemons, services, and mount points [*citation needed*].

systemd Cheat Sheet

VIEWING systemd INFORMATION

`systemctl list-dependencies`	Show a unit's dependencies
`systemctl list-sockets`	List sockets and what activates
`systemctl list-jobs`	View active systemd jobs
`systemctl list-unit-files`	See unit files and their states
`systemctl list-units`	Show if units are loaded/active
`systemctl get-default`	List default target (like run level)

WORKING WITH SERVICES

`systemctl stop` *service*	Stop a running service
`systemctl start` *service*	Start a service
`systemctl restart` *service*	Restart a running service
`systemctl reload` *service*	Reload all config files in service
`systemctl daemon-reload`	Must run to reload changed unit files
`systemctl status` *service*	See if service is running/enabled
`systemctl --failed`	Shows services that failed to run
`systemctl reset-failed`	Resets any units from failed state
`systemctl enable` *service*	Enable a service to start on boot
`systemctl disable` *service*	Disable service--won't start at boot
`systemctl show` *service*	Show properties of a service (or other unit)
`systemctl edit` *service*	Create snippit to drop in unit file
`systemctl edit --full` *service*	Edit entire unit file for service
`systemctl -H` *host* `status network`	Run any systemctl command remotely

CHANGING SYSTEM STATES

`systemctl reboot`	Reboot the system (reboot.target)
`systemctl poweroff`	Power off the system (poweroff.target)
`systemctl emergency`	Put in emergency mode (emergency.target)
`systemctl default`	Back to default target (multi-user.target)

VIEWING LOG MESSAGES

`journalctl`	Show all collected log messages
`journalctl -u network.service`	See network service messages
`journalctl -f`	Follow messages as they appear
`journalctl -k`	Show only kernel messages

USING UNIT FILES

Besides **services**, most systemd commands can work with these unit types: **paths**, **slices**, **snapshots**, **sockets**, **swaps**, **targets**, and **timers**

RHEL_482736_1118

Copyright © 2018 Red Hat, Inc. Red Hat, Red Hat Enterprise Linux, the Shadowman logo, and JBoss are trademarks of Red Hat, Inc., registered in the U.S. and other countries. Linux® is the registered trademark of Linus Torvalds in the U.S. and other countries.

redhat.

Hello World!

Let’s try running the “hello-world” image:

- Currently we don’t have any images on our local cache, if an image is not available on the cache, docker automatically downloads it from docker-hub on internet.
- What the hello-world image does is just showing the message shown on the figure; nevertheless, it contains a full linux environment which uses your kernel but has a user space of its own.

```
hayral@Computer1:~$
hayral@Computer1:~$ sudo docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
b8dfde127a29: Pull complete
Digest: sha256:f2266cbfc127c960fd30e76b7c792dc23b588c0db76233517e1891a4e357d519
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

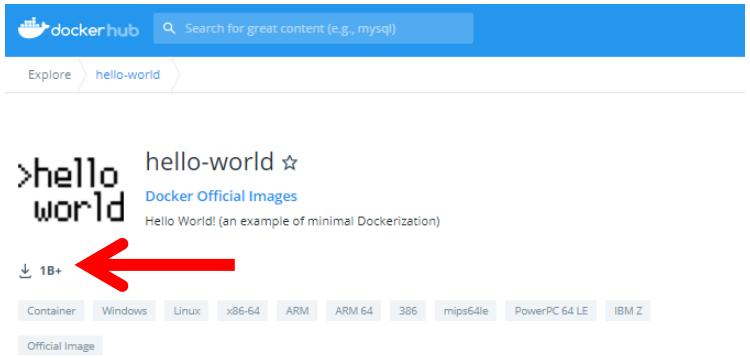
To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

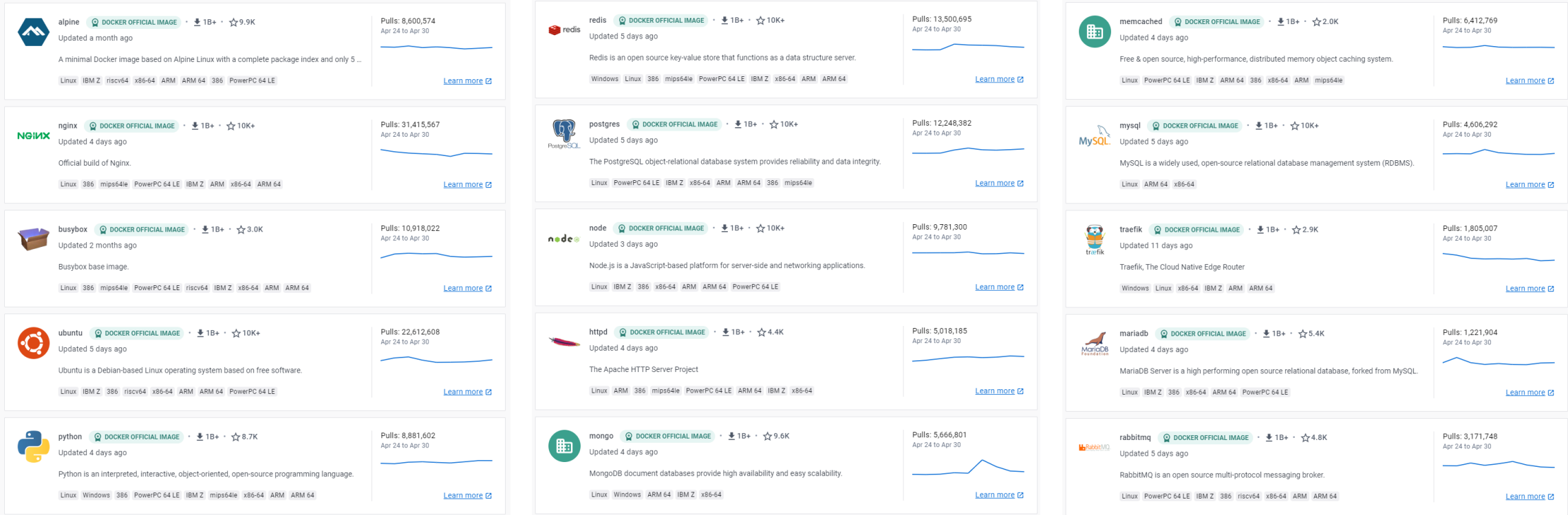
For more examples and ideas, visit:
 https://docs.docker.com/get-started/

hayral@Computer1:~$
```

Hello World!



- This official hello-world image has been downloaded more than one billion times!
- List below shows most used docker images as of May 2023. Check this list from DockerHub because it also shows which skills are currently valuable on job market!



Detailed Version Information

- You can check the version of different components of Docker with `docker version`
- You can see that it consists of a client and a server, furthermore the server consists of components like
  - Engine
  - containerd
  - runc
  - docker-init

```
hayral@Computer1:~$
hayral@Computer1:~$ sudo docker version
Client:
 Version:           20.10.2
 API version:       1.41
 Go version:        go1.13.8
 Git commit:        20.10.2-0ubuntu1~20.04.2
 Built:             Tue Mar 30 21:24:57 2021
 OS/Arch:           linux/amd64
 Context:           default
 Experimental:      true

Server:
 Engine:
  Version:          20.10.2
  API version:      1.41 (minimum version 1.12)
  Go version:       go1.13.8
  Git commit:       20.10.2-0ubuntu1~20.04.2
  Built:            Mon Mar 29 19:10:09 2021
  OS/Arch:          linux/amd64
  Experimental:     false
 containerd:
  Version:          1.3.3-0ubuntu2.3
  GitCommit:
 runc:
  Version:          spec: 1.0.2-dev
  GitCommit:
 docker-init:
  Version:          0.19.0
  GitCommit:
hayral@Computer1:~$
```

More info

- You can get more detailed info about the drivers, installed plugins and versions of the components by issuing the `docker info` command.

```
hayral@Computer1:~$ sudo docker info
Client:
 Context:    default
 Debug Mode: false

Server:
 Containers: 1
  Running: 0
  Paused: 0
  Stopped: 1
 Images: 1
 Server Version: 20.10.2
 Storage Driver: overlay2
  Backing Filesystem: extfs
  Supports d_type: true
  Native Overlay Diff: true
 Logging Driver: json-file
 Cgroup Driver: cgroupfs
 Cgroup Version: 1
 Plugins:
  Volume: local
  Network: bridge host ipvlan macvlan null overlay
  Log: awslogs fluentd gcplogs gelf journald json-file local logentries splunk syslog
 Swarm: inactive
 Runtimes: io.containerd.runc.v2 io.containerd.runtime.v1.linux runc
 Default Runtime: runc
 Init Binary: docker-init
 containerd version:
 runc version:
 init version:
 Security Options:
  apparmor
  seccomp
   Profile: default
 Kernel Version: 5.4.0-58-generic
 Operating System: Linux Mint 20.1
 OSType: linux
 Architecture: x86_64
 CPUs: 2
 Total Memory: 855.1MiB
 Name: Computer1
 ID: IRUL:MOBE:MGSP:I3AH:TTBZ:NO7U:5U7Q:I6OW:2JCT:UKJE:HFE2:QVSI
 Docker Root Dir: /var/lib/docker
 Debug Mode: false
 Registry: https://index.docker.io/v1/
 Labels:
 Experimental: false
 Insecure Registries:
  127.0.0.0/8
 Live Restore Enabled: false
```

Docker Commands

- Docker is administered with many commands which you specify as a command option of docker command (the use of the term command in two contexts is a little confusing you pass the name of a “docker command” as a command option to “docker” which is the command to manage docker.)

```
docker <options> command
```

- There are two kinds of docker commands
  - management commands and
  - regular commands (listed on the next slide)

```
hayral@Computer1:~$ docker help

Usage:  docker [OPTIONS] COMMAND

A self-sufficient runtime for containers

Options:
      --config string      Location of client config files (default "/home/hayral/.docke
  -c, --context string     Name of the context to use to connect to the daemon (override
                           and default context set with "docker context use")
  -D, --debug              Enable debug mode
  -H, --host list          Daemon socket(s) to connect to
  -l, --log-level string   Set the logging level ("debug"|"info"|"warn"|"error"|"fatal")
      --tls                Use TLS; implied by --tlsverify
      --tlscacert string   Trust certs signed only by this CA (default "/home/hayral/.do
      --tlscert string     Path to TLS certificate file (default "/home/hayral/.docker/c
      --tlskey string      Path to TLS key file (default "/home/hayral/.docker/key.pem")
      --tlsverify          Use TLS and verify the remote
  -v, --version            Print version information and quit

Management Commands:
  builder     Manage builds
  config      Manage Docker configs
  container   Manage containers
  context     Manage contexts
  image       Manage images
  manifest    Manage Docker image manifests and manifest lists
  network     Manage networks
  node        Manage Swarm nodes
  plugin      Manage plugins
  secret      Manage Docker secrets
  service     Manage services
  stack       Manage Docker stacks
  swarm       Manage Swarm
  system      Manage Docker
  trust       Manage trust on Docker images
  volume      Manage volumes
```

Docker Commands

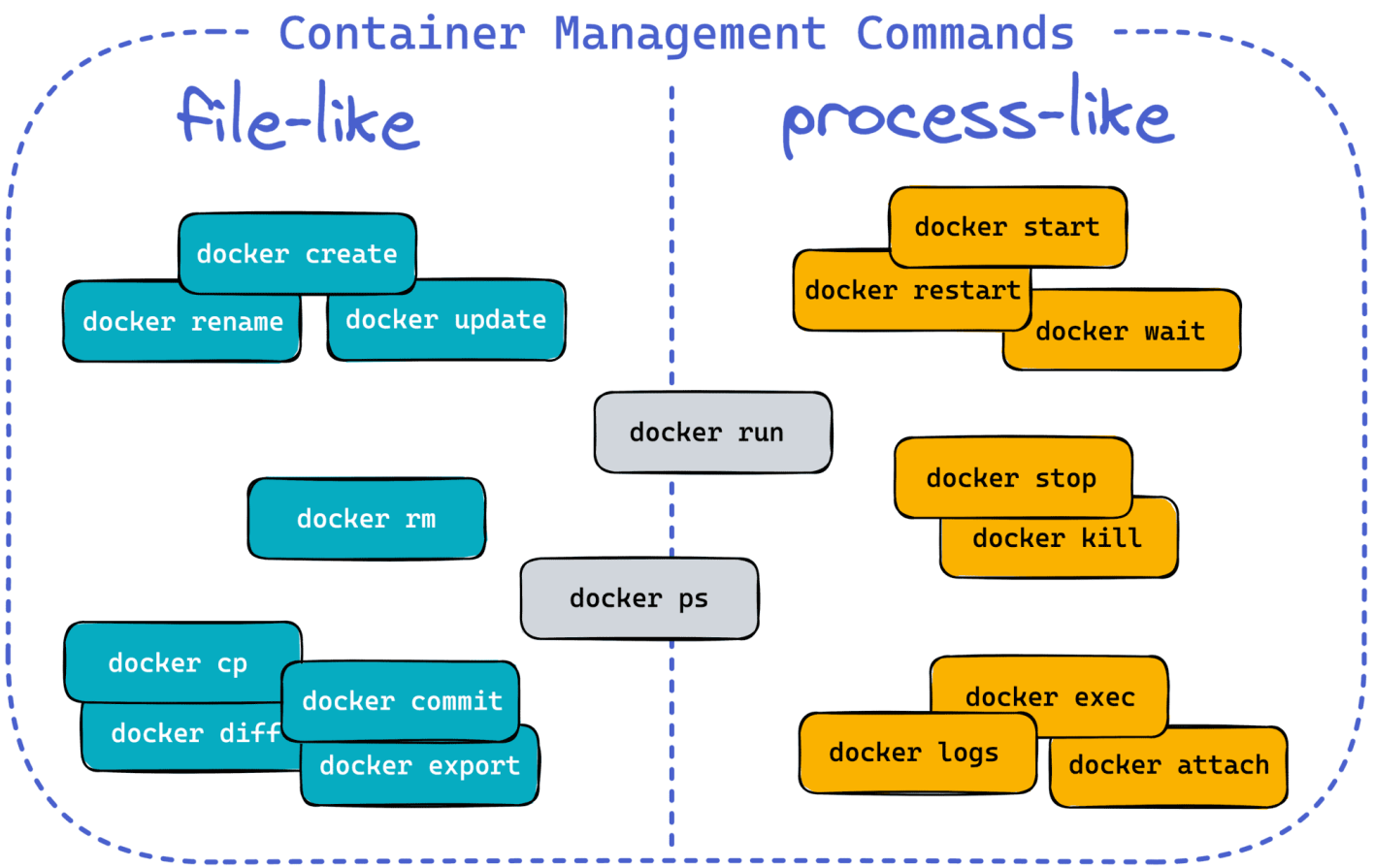
- Regular docker commands allow you to
  - create/list/delete the containers/images
  - start/stop/pause/resume/kill running containers
  - connect to the console of a running container
  - send shell commands to a running container
  - copy files from/to container's internal file system
  - etc..

```
Commands:
  attach      Attach local standard input, output, and error streams to a running container
  build       Build an image from a Dockerfile
  commit      Create a new image from a container's changes
  cp          Copy files/folders between a container and the local filesystem
  create      Create a new container
  diff        Inspect changes to files or directories on a container's filesystem
  events      Get real time events from the server
  exec        Run a command in a running container
  export      Export a container's filesystem as a tar archive
  history     Show the history of an image
  images      List images
  import      Import the contents from a tarball to create a filesystem image
  info        Display system-wide information
  inspect     Return low-level information on Docker objects
  kill        Kill one or more running containers
  load        Load an image from a tar archive or STDIN
  login       Log in to a Docker registry
  logout      Log out from a Docker registry
  logs        Fetch the logs of a container
  pause       Pause all processes within one or more containers
  port        List port mappings or a specific mapping for the container
  ps          List containers
  pull        Pull an image or a repository from a registry
  push        Push an image or a repository to a registry
  rename      Rename a container
  restart     Restart one or more containers
  rm          Remove one or more containers
  rmi         Remove one or more images
  run         Run a command in a new container
  save        Save one or more images to a tar archive (streamed to STDOUT by default)
  search      Search the Docker Hub for images
  start       Start one or more stopped containers
  stats       Display a live stream of container(s) resource usage statistics
  stop        Stop one or more running containers
  tag         Create a tag TARGET_IMAGE that refers to SOURCE_IMAGE
  top         Display the running processes of a container
  unpause     Unpause all processes within one or more containers
  update      Update configuration of one or more containers
  version     Show the Docker version information
  wait        Block until one or more containers stop, then print their exit codes

Run 'docker COMMAND --help' for more information on a command.

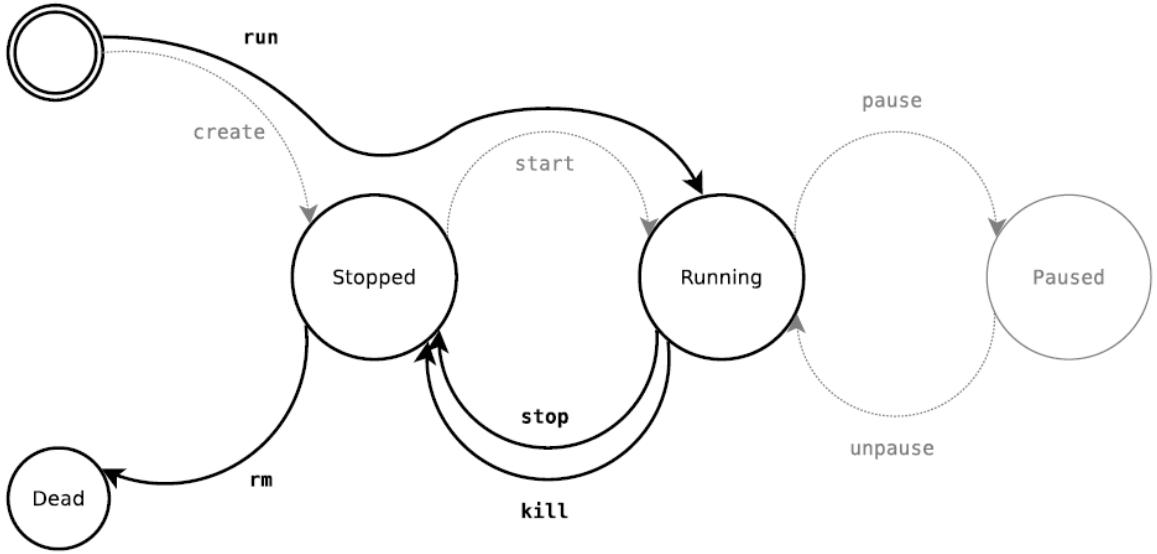
To get more help with docker, check out our guides at https://docs.docker.com/go/guides/
hayral@Computer1:~$
```

Docker Commands



Container Management Commands & States

command	description
`docker create` *image* [*command*] `docker run` *image* [*command*]	create the container = `create` + `start`
`docker start` *container...* `docker stop` *container...* `docker kill` *container...* `docker restart` *container...*	start the container graceful[2] stop kill (SIGKILL) the container = `stop` + `start`
`docker pause` *container...* `docker unpause` *container...*	suspend the container resume the container
`docker rm` [`-f`[3]] *container...*	destroy the container



[2] send SIGTERM to the main process + SIGKILL 10 seconds later
[3] `-f` allows removing running containers (= `docker kill` + `docker rm`)

Inspecting & Interacting with the containers

command	description
`docker ps`	list running containers
`docker ps -a`	list all containers
`docker logs` [`-f`[6]] *container*	show the container output (*stdout+stderr*)
`docker top` *container* [*ps options*]	list the processes running inside the containers
`docker diff` *container*	show the differences with the image (modified files)
`docker inspect` *container...*	show low-level infos (in json format)

command	description
`docker attach` *container*	attach to a running container (stdin/stdout/stderr)
`docker cp` *container:path hostpath*|- `docker cp` *hostpath*|- *container:path*	copy files from the container copy files into the container
`docker export` *container*	export the content of the container (tar archive)
`docker exec` *container args...*	run a command in an existing container (**useful** for debugging)
`docker wait` *container*	wait until the container terminates and return the exit code
`docker commit` *container image*	commit a new docker image (snapshot of the container)

Image Management & Transfer Commands

command	description
`docker images`	list all local images
`docker history` *image*	show the image history (list of ancestors)
`docker inspect` *image...*	show low-level infos (in json format)
`docker tag` *image tag*	tag an image
`docker commit` *container image*	create an image (from a container)
`docker import` *url*|- [*tag*]	create an image (from a tarball)
`docker rmi` *image...*	delete images

Using the registry API

`docker pull` *repo*[:*tag*]...	pull an image/repo from a registry
`docker push` *repo*[:*tag*]...	push an image/repo from a registry
`docker search` *text*	search an image on the official registry
`docker login` ...	login to a registry
`docker logout` ...	logout from a registry

Manual transfer

`docker save` *repo*[:*tag*]...	export an image/repo as a tarbal
`docker load`	load images from a tarball
`docker-ssh`[10] ...	proposed script to transfer images between two daemons over ssh

Cheatsheet for Docker CLI

Run a new Container

Start a new Container from an Image
```
docker run IMAGE
docker run nginx
```

...and assign it a name
```
docker run --name CONTAINER IMAGE
docker run --name web nginx
```

...and map a port
```
docker run -p HOSTPORT:CONTAINERPORT IMAGE
docker run -p 8080:80 nginx
```

...and map all ports
```
docker run -P IMAGE
docker run -P nginx
```

...and start container in background
```
docker run -d IMAGE
docker run -d nginx
```

...and assign it a hostname
```
docker run --hostname HOSTNAME IMAGE
docker run --hostname srv nginx
```

...and add a dns entry
```
docker run --add-host HOSTNAME:IP IMAGE
```

...and map a local directory into the container
```
docker run -v HOSTDIR:TARGETDIR IMAGE
docker run -v ~/:/usr/share/nginx/html nginx
```

...but change the entrypoint
```
docker run -it --entrypoint EXECUTABLE IMAGE
docker run -it --entrypoint bash nginx
```

Manage Containers

Show a list of running containers
```
docker ps
```

Show a list of all containers
```
docker ps -a
```

Delete a container
```
docker rm CONTAINER
docker rm web
```

Delete a running container
```
docker rm -f CONTAINER
docker rm -f web
```

Delete stopped containers
```
docker container prune
```

Stop a running container
```
docker stop CONTAINER
docker stop web
```

Start a stopped container
```
docker start CONTAINER
docker start web
```

Copy a file from a container to the host
```
docker cp CONTAINER:SOURCE TARGET
docker cp web:/index.html index.html
```

Copy a file from the host to a container
```
docker cp TARGET CONTAINER:SOURCE
docker cp index.html web:/index.html
```

Start a shell inside a running container
```
docker exec -it CONTAINER EXECUTABLE
docker exec -it web bash
```

Rename a container
```
docker rename OLD_NAME NEW_NAME
docker rename 096 web
```

Create an image out of container
```
docker commit CONTAINER
docker commit web
```

Manage Images

Download an image
```
docker pull IMAGE[:TAG]
docker pull nginx
```

Upload an image to a repository
```
docker push IMAGE
docker push myimage:1.0
```

Delete an image
```
docker rmi IMAGE
```

Show a list of all Images
```
docker images
```

Delete dangling images
```
docker image prune
```

Delete all unused images
```
docker image prune -a
```

Build an image from a Dockerfile
```
docker build DIRECTORY
docker build .
```

Tag an image
```
docker tag IMAGE NEWIMAGE
docker tag ubuntu ubuntu:18.04
```

Build and tag an image from a Dockerfile
```
docker build -t IMAGE DIRECTORY
docker build -t myimage .
```

Save an image to .tar file
```
docker save IMAGE > FILE
docker save nginx > nginx.tar
```

Load an image from a .tar file
```
docker load -i TARFILE
docker load -i nginx.tar
```

Info & Stats

Show the logs of a container
```
docker logs CONTAINER
docker logs web
```

Show stats of running containers
```
docker stats
```

Show processes of container
```
docker top CONTAINER
docker top web
```

Show installed docker version
```
docker version
```

Get detailed info about an object
```
docker inspect NAME
docker inspect nginx
```

Show all modified files in container
```
docker diff CONTAINER
docker diff web
```

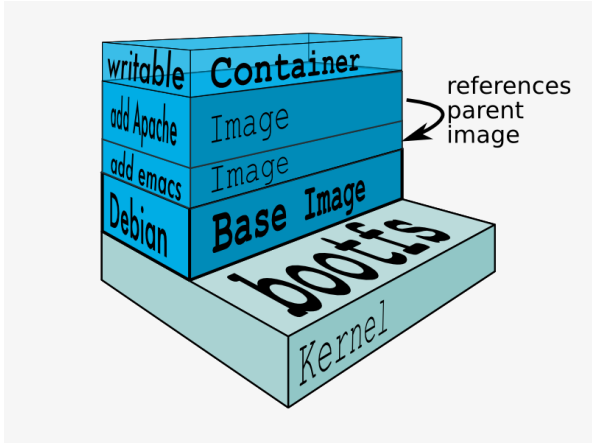
Show mapped ports of a container
```
docker port CONTAINER
docker port web
```

What is a Docker Image/Container ?

A docker image is an immutable snapshot of the filesystem

A docker container is

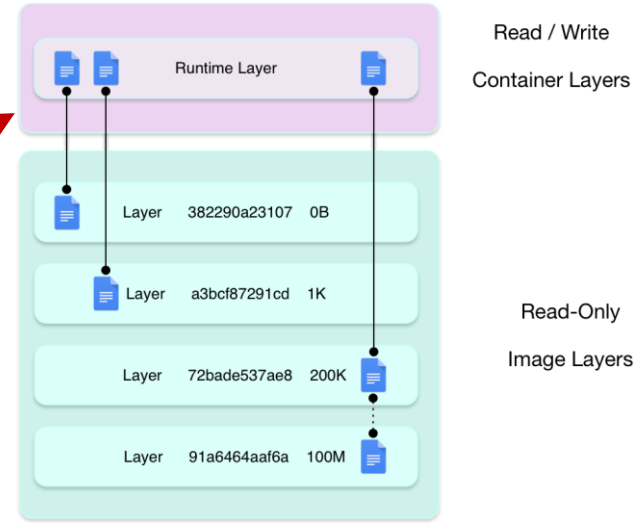
- a temporary file system
  - layered over an immutable fs (docker image)
  - fully writable (copy-on-write[1])
  - dropped at container's end of life (unless a `commit` is made)
- a network stack
  - with its own private address *(by defaut in 172.17.x.x)*
- a process group
  - one main process launched inside the container
  - all sub-processes SIGKILLed when the main process exits

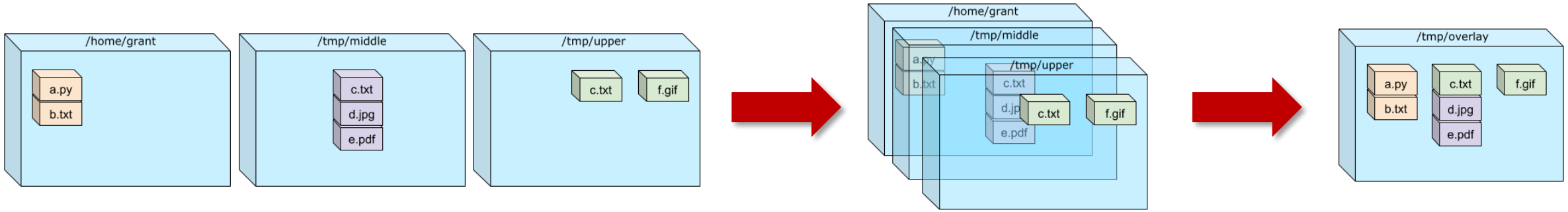


[1] several possible methods: overlayfs (default), btrfs, lvm, zfs, aufs

How layered file systems work?

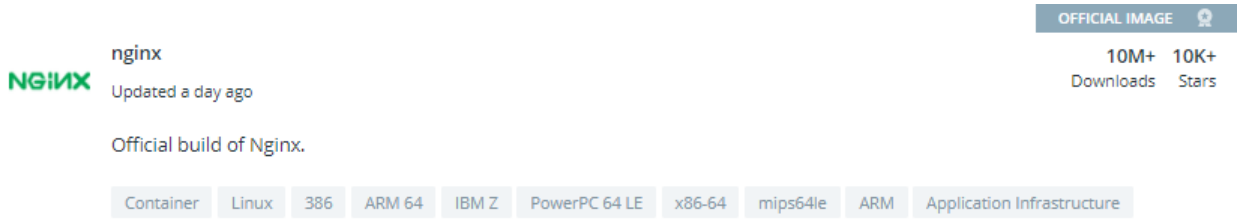
- Image: Layers of files stacked on top of each other. You view the final state of the file system when looked from the top, where changes from higher layers override the lower layers
- Container: When a container is created from an image, **a temporary layer** is added on top which accumulates all the changes made during runtime, but because it is temporary all changes will be lost when container is removed.
  - You can map an external path to a path inside container and use that location to store persistent data.





Another image, another container

- Nginx is a web server, load balancer and reverse proxy.
- It is the 2nh most downloaded image on Docker Hub (as of May 2023):

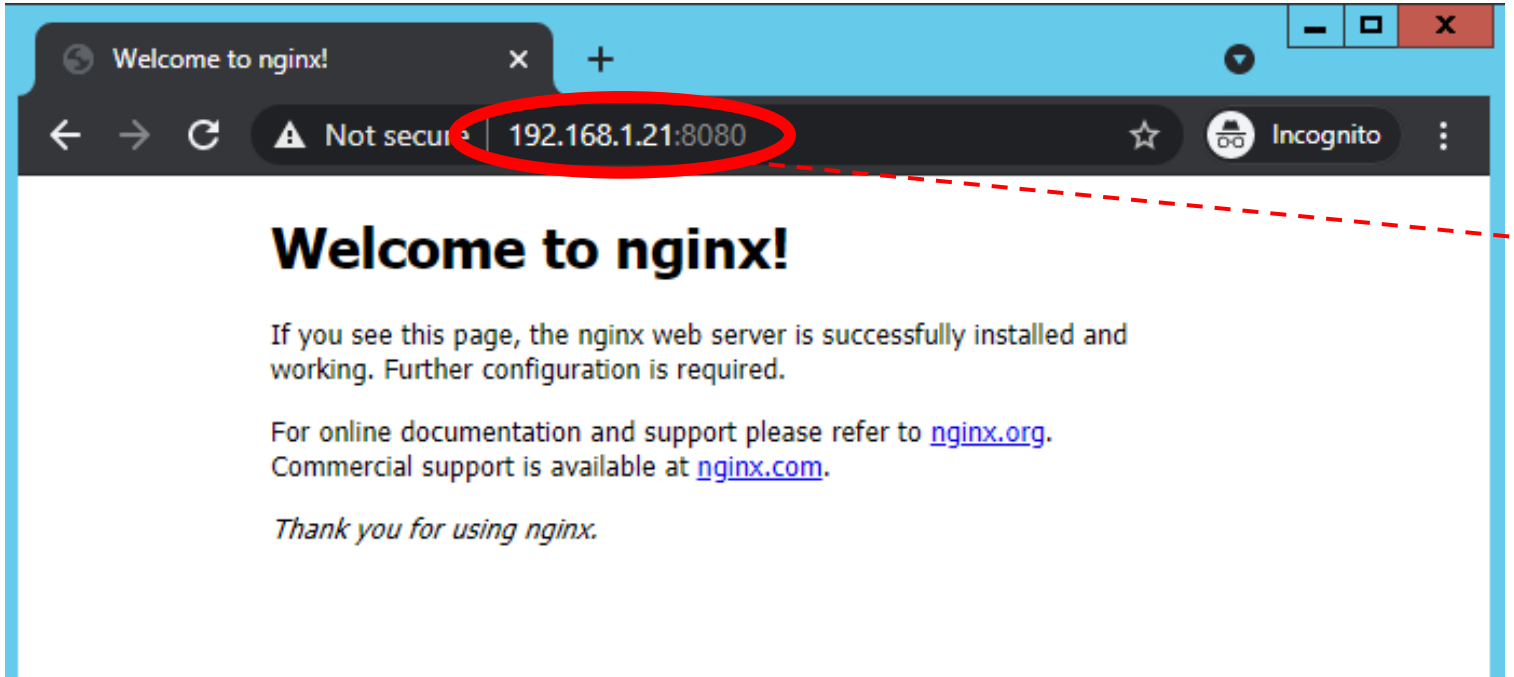


- You can run a container with this image using the command below (*we will look at command options in detail on later slides*):

```
hayral@Computer1:~$ sudo docker container run -d --name nginx-test -p 8080:80 nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
f7ec5a41d630: Pull complete
aa1efa14b3bf: Pull complete
b78b95af9b17: Pull complete
c7d6bca2b8dc: Pull complete
cf16cd8e71e0: Pull complete
0241c68333ef: Pull complete
Digest: sha256:75a55d33ecc73c2a242450a9f1cc858499d468f077ea942867e662c247b5e412
Status: Downloaded newer image for nginx:latest
4d0c949313178273c381ae35623d9c80dfc054bd95035a2c630b5939f0d93b2d
hayral@Computer1:~$
```

Services exposed by container

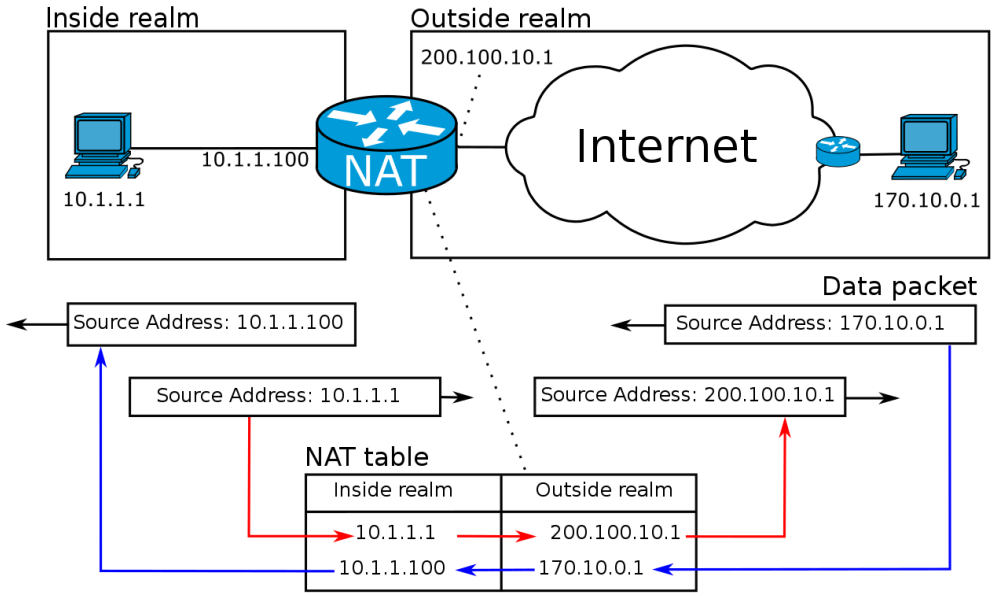
- Congratulations! You have just installed a new web server.
- You can connect to the Nginx server running inside the container by using a browser on Host OS and connect to the port 8080 of the Guest OS.
  - the end point <**containerIP:80**> is mapped to <**guestOSIP:8080**> with the command parameter **`-p 8080:80`** on docker command
  - Why we need mapping? → Docker and browser are on different networks.



Network Boundary

- When two networks have a **boundary** (*a device or devices which connects to both*) the two networks can communicate with each other by using the device on the boundary as a **router**.

- **NAT** (network address translation) allows multiple machines on an internal network to communicate with a different outer i.e. LAN (**private IP**) ←→ Internet (**public IP**/real)



Your current setup

- For the following slides we are going to assume the following three network setup which is currently used by the majority of you:

- A Windows computer (*Host OS*) running on the physical computer.
  Network: LAN, network range: 192.168.1.X

- A Linux Virtual Machine (*Guest OS*) running on top of Host OS.
  Network: depends on adapter type **bridged adapter** (→ part of LAN) or **NAT** (→ VirtualBox virtual network)

- Docker running on top of Guest OS.
  Network: Docker virtual network, network range: 172.17.X.X

What is Network Address Translation (NAT)?

- **NAT** allows multiple machines on an internal network to communicate with a different outer network by translating the addresses from one to the next at the **boundary** to provide transparent communication.
  - It does this by keeping track of incoming and outgoing traffic in a table of the form <src-ip:src-port>,<target-ip:target-port> to translate and map internal address/port pairs to external address/port pairs.
- There are two **boundaries** on the previous slide:
  - Docker NAT is at the boundary of Docker Network (its internal network) and the network outside (LAN if you run it directly, in our case VirtualBox's network because we run it on VM)
    - Docker provides NAT between your **Guest OS** and **Containers**.
  - VirtualBox is at the boundary of VirtualBox virtual network (**Guest OS** network, i.e. network as seen by the VM) and **Host OS** network (i.e. your LAN)
    - VirtualBox provides NAT between your Host OS and Guest OS.
    - *if you set connection settings to **bridged connection** and not NAT then VirtualBox will use the same network as Host (it'll be part of your LAN) and it will be visible to other computers on your LAN.*
- Actually there is a 3rd boundary: Your ADSL modem is at the **boundary** of your **LAN** and **Internet**, and it provides NAT between them.
- Different networks use different types of IP addresses:

  Internet use **public IP** addresses. ←→ Local networks use **private IP** addresses.

Three networks, three IP addresses

- Host OS IP address
  - This is the IP address of your main OS (the outer most OS, most probably a Windows)
    - The IP address for this is most probably assigned by **DHCP** automatically (i.e. by your ADSL modem if you are at home) unless you specifically assigned a number manually.
- Guest OS (VM) IP address
  - This one depends on your VM settings;
    - if you set it to **NAT** the internal DHCP server of VirtualBox will assign an IP address automatically.
    - if you set it to **bridged connection** (and you are not connected with wireless 😕) it will try to get an IP from a **DHCP** server it one is available on your LAN (*almost all ADSL modems also act as a **DHCP** server*), otherwise it will require you to assign an IP address manually.
- Container (running inside VM) IP address
  - This is assigned automatically by Docker; but you can access with a host name which acts similar to DNS.

What is DHCP?

- The Dynamic Host Configuration Protocol (DHCP) is a network management protocol used on IP networks for automatically assigning IP addresses and other communication parameters to devices connected to the network using a client–server architecture.

- DHCP eliminates the need for individually configuring network devices manually.



Public vs Private IP addresses

- A **public IP** address is a globally routable unicast IP address, meaning that the address is not an address reserved for use in private networks.
  - Public IP addresses are used for communication between hosts on the global Internet.
- Not all computers connected to the Internet need a **public IP** address.
  - **Private networks** are widely used and typically connect to the Internet with network address translation (NAT), when needed.
  - In a home network/LAN, computers use **private IP** addresses for local communication, and they share a **public IP** address assigned by the ISP to their .
- Most public IP addresses on domestic use (*i.e. not used on a server or business setting*) change relatively often.
    - Any type of IP address that changes is called a **dynamic IP** address.
    - In home networks, the ISP usually assigns a **dynamic IP**.
- There are three non-overlapping ranges of IPv4 addresses reserved for **private networks**.
    - These addresses are not routed on the Internet and thus their use need not be coordinated with an IP address registry.
    - Any user may use any of the reserved blocks.
    - A network administrator can further divide a block into subnets;
      - for example, many home routers automatically use the default address range of 192.168.1.0/24.

	Private address range	
Class	start address	finish address
A	10.0.0.0	10.255.255.255
B	172.16.0.0	172.31.255.255
C	192.168.0.0	192.168.255.255

	Public address range	
Class	start address	finish address
A	0.0.0.0	126.255.255.255
B	128.0.0.0	191.255.255.255
C	192.0.0.0	223.255.255.255
D	224.0.0.0	239.255.255.255
E	240.0.0.0	254.255.255.255

Public IP vs Private IP	
Public IP	*Private IP*
Used over the Public Network ex. WAN	Used with in the private network ex. LAN
Recognized over the Internet	Not recognized over the Internet
Public IP are unique over the Globe.	Private IP are unique with in the network or LAN.
Public IP are paid.	Private IP are free of cost.
Assigned by Network Administrator.	Assigned by Internet Service Provider /IANA

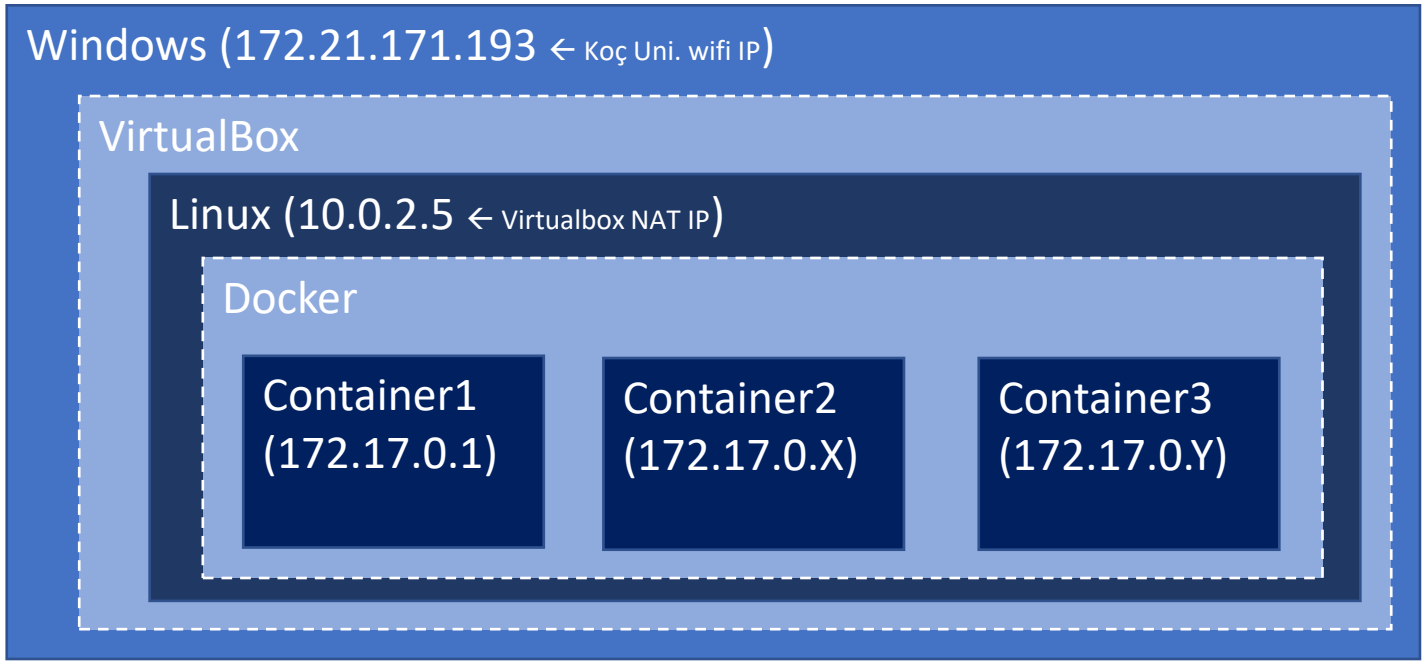
IP Addresses

- At this point we are dealing with 3 different networks and 3 different IP addresses on our machine:
  - LAN / Host OS IP address
  - Network of Virtual Machine / Guest OS (VM) IP address
  - Network of Container (running inside VM) / Container IP address

```
hakan@hakan-main-host:~$ ifconfig
docker0: flags=4099<UP,BROADCAST,MULTICAST>  mtu 1500
        inet 172.17.0.1  netmask 255.255.0.0  broadcast 172.17.255.255
        ether 02:42:1f:5e:e8:5b  txqueuelen 0  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

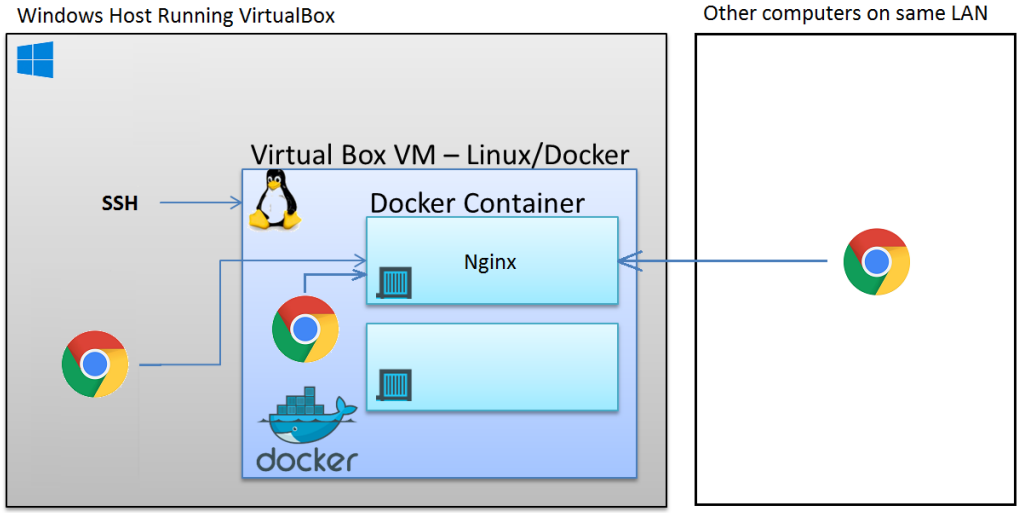
enp0s3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.0.2.5  netmask 255.255.255.0  broadcast 10.0.2.255
        inet6 fe80::28:614e:a925:d602  prefixlen 64  scopeid 0x20<link>
        ether 08:00:27:e3:4a:2f  txqueuelen 1000  (Ethernet)
        RX packets 47  bytes 8430 (8.4 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 100  bytes 11297 (11.2 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 171  bytes 18643 (18.6 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 171  bytes 18643 (18.6 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```



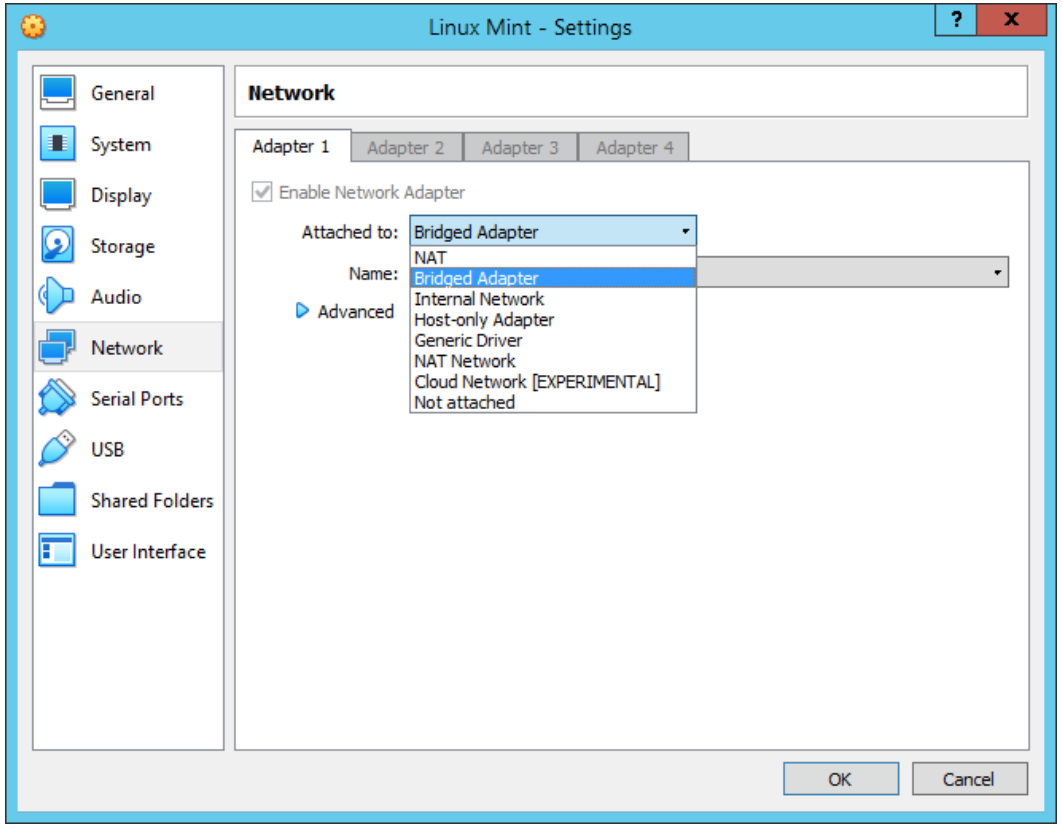
Different locations to connect from

- You can connect to the services exposed by docker from multiple locations and each time the target IP may or may not be different.
  1. If you try to connect to the container from within the guest OS (VM) you have to enter the IP of the container.
     - If we set the command parameter `-p 8080:80` then docker maps the port 80 of container to port 8080 of VM and we can use the guest OS (VM) IP instead of container IP.
     - Otherwise the NAT functionality of Docker will allow us to use the container IP on guest OS by translating between the two networks.
  2. If you try to connect to the container from the host OS you have to enter the IP address of the guest OS (VM) and the container inside must have its port mapped to guest OS as mentioned above.
     - If you choose bridged adapter on Virtual Machine settings then the IP address of guest OS will be in the same address space as host OS, otherwise it will be in a different address space and NAT functionality of VirtualBox will translate between these two networks.
  3. If you try to connect to this container from a different container on the same Docker internal network you have to use the internal IP assigned by Docker to this container.
     - Docker allows you to assign a network alias to each container so that each container can access each other by name, without needing to know the automatically assigned IP to the containers by the internal DHCP of Docker.



VirtualBox Network Settings

- If you choose **NAT network**
  - VirtualBox will create a virtual network for your Virtual Machine which is different from your host OS network (i.e. LAN).
  - These networks will have IP spaces of their own, and VirtualBox will act as a translation layer between the two.
- If you choose **Bridged Adapter**
  - your Virtual Machine will act just like another machine which is directly connected to the same network as your host.
  - They will share the same IP space, therefore if you assign an IP to your VM which is already used on your LAN you'll get an IP conflict.



	VM ↔ VM	VM → Host	VM ← Host	VM → LAN	VM ← LAN
Not attached	–	–	–	–	–
NAT	–	+	Port Forward	+	Port Forward
NAT Network	+	+	Port Forward	+	Port Forward
Bridged	+	+	+	+	+
Internal Network	+	–	–	–	–
Host-only	+	+	+	–	–

Different VM Network Options

- There are two different NAT zones on the diagram to the right.
  - The first one is behind the router/firewall which uses the private address range 192.168.X.X.
    - The NAT address translation is performed by the firewall/router.
    - Ina domestic settings the ADSL modem provides the router, firewall, NAT, local DNS and local DHCP services.
  - The second one is provided by VirtualBox to the VMs (private IP range 172.176.1.X)
  - If you run Docker on the VMs, the Docker will provide a third layer of Network Address Translation
- Observer the difference between the Virtual Adapter which provides bridged networking and NAT

